第23课 高精度加法

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：基础高精度加法（标准模板）

题目：输入两个非负整数 a 和 b（位数可达 500 位），输出 a + b。

讲解思路：

1. 两个数都非常大，不能用 `int` / `long long`，必须用字符串读入。
2. 字符串倒序转成整数数组，便于从低位开始加。
3. 逐位相加，处理进位。
4. 去前导零，倒序输出。

完整代码：

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
 // 第 1 步：字符串读入
 string s1, s2;
 cin >> s1 >> s2;

 // 准备数组：a 存第一个数，b 存第二个数，c 存结果
 // 数组开大一点，510 够用了（最多 500 位 + 进位）
 int a[510] = {0};
 int b[510] = {0};
 int c[510] = {0};

 int len1 = s1.size(); // 第一个数的位数
 int len2 = s2.size(); // 第二个数的位数

 // 第 2 步：字符串倒序存入数组
 // s1 = "386"，那么 s1[0]='3', s1[1]='8', s1[2]='6'
 // 我们要让 a[0]=6, a[1]=8, a[2]=3（个位在前）
 for (int i = 0; i < len1; i++) {
 a[i] = s1[len1 - 1 - i] - '0';
 }
 for (int i = 0; i < len2; i++) {
 b[i] = s2[len2 - 1 - i] - '0';
 }

 // 第 3 步：逐位相加
 int len = max(len1, len2); // 取较长的位数
 int carry = 0; // 进位，一开始是 0

 for (int i = 0; i < len; i++) {
 int sum = a[i] + b[i] + carry; // 当前位的和 = 两个数的这一位 + 进位
 c[i] = sum % 10; // 当前位的结果（取个位）
 carry = sum / 10; // 新的进位（取十位）
 }

 // 第 4 步：如果最高位还有进位，别忘了！
 if (carry > 0) {
 c[len] = carry;
 len++; // 结果多了一位
 }

 // 第 5 步：倒序输出
 for (int i = len - 1; i >= 0; i--) {
 cout << c[i];
 }
 cout << endl;
```

```
 return 0;
}
```

课堂演示：

```
输入：
9876
2345
输出：
12221
```

```
输入：
999
1
输出：
1000
```

> **老师强调：**
> - `s1[len1 - 1 - i] - '0'` 这里的 `- '0'` 是把字符变成数字，还记得 ASCII 码吗？字符 `'3'` 的编号是 51，数字 `3` 是 `51 - 48 = 3`，而 `'0'` 的编号就是 48。所以 `'3' - '0' = 3`。
> - `carry`（进位）就像是你做竖式时写在上面的那个小小的 "1"。
> - `sum % 10` 留下个位写在当前格子，`sum / 10` 就是要进到下一位的数。

例题 2：两个数位数差距很大

题目： 计算 `10000000000000000000`（1 后面 20 个 0）加 `1`。

讲解思路：

> 这道题考验的是：两个数位数差很多的时候，程序还能不能正确运行？
> 答案是：当然可以！因为我们数组初始化为全 0，短的那个数后面的高位自动是 0，加上去不影响结果。

课堂运行：

```
输入：
10000000000000000000
1
输出：
10000000000000000001
```

> 老师引导学生思考：
> - 如果用 `long long` 算这道题会怎样？（答：`long long` 存不下 21 位的数，会出错！）
> - 但我们的高精度代码轻轻松松就搞定了，是不是很厉害？

例题 3：连续进位的情况

题目：计算 `999999` + `1`。

讲解思路：

> 这道题是"连续进位"，从个位一直进到最高位还要再进一位。就像多米诺骨牌一样，一个接一个倒下去。

课堂推演（和学生一起走一遍循环）：

```
i=0: a[0]=9, b[0]=1, carry=0 sum=10, c[0]=0, carry=1
i=1: a[1]=9, b[1]=0, carry=1 sum=10, c[1]=0, carry=1
i=2: a[2]=9, b[2]=0, carry=1 sum=10, c[2]=0, carry=1
i=3: a[3]=9, b[3]=0, carry=1 sum=10, c[3]=0, carry=1
i=4: a[4]=9, b[4]=0, carry=1 sum=10, c[4]=0, carry=1
i=5: a[5]=9, b[5]=0, carry=1 sum=10, c[5]=0, carry=1
循环结束后 carry=1，所以 c[6]=1，len 变成 7。
倒序输出：1000000
```

> **老师强调**：循环结束后**一定要检查 carry**！这是很多同学容易忘掉的地方。忘了的话，999999 + 1 就只会输出 000000，结果完全错了！

附：例题源代码与运行结果

例题 1：Basic High-Precision Addition (Standard Template)

```
// Read two non-negative integers (up to 500 digits) and print their sum.
#include <iostream>
#include <cstring>
using namespace std;

int main() {
 // Step 1: Read two large numbers as strings
 string s1, s2;
 cin >> s1 >> s2;

 // Arrays to store digits: a for first number, b for second, c for result
 // Size 510 is enough (500 digits + possible carry)
 int a[510] = {0};
 int b[510] = {0};
 int c[510] = {0};

 int len1 = s1.size(); // Number of digits in first number
 int len2 = s2.size(); // Number of digits in second number

 // Step 2: Store string digits into arrays in reverse order
 // e.g., s1 = "386" -> a[0]=6, a[1]=8, a[2]=3 (units digit first)
 for (int i = 0; i < len1; i++) {
 a[i] = s1[len1 - 1 - i] - '0';
 }
 for (int i = 0; i < len2; i++) {
 b[i] = s2[len2 - 1 - i] - '0';
 }

 // Step 3: Add digit by digit
 int len = max(len1, len2); // Use the longer length
 int carry = 0; // Carry starts at 0

 for (int i = 0; i < len; i++) {
 int sum = a[i] + b[i] + carry; // Current digit sum + carry
 c[i] = sum % 10; // Keep the ones digit
 carry = sum / 10; // New carry
 }

 // Step 4: Don't forget the final carry!
 if (carry > 0) {
 c[len] = carry;
 len++; // Result has one more digit
 }

 // Step 5: Print result in reverse order
 for (int i = len - 1; i >= 0; i--) {
 cout << c[i];
 }
 cout << endl;

 return 0;
}
```

Sample Input

```
9876
2345
```

Expected Output

```
12221
```

例题 2：Two Numbers with Very Different Lengths

```
// Compute 100000000000000000000 (1 followed by 20 zeros) + 1.
// This uses the same code as Example 1. The shorter number's higher
// digits are automatically 0 because the array is initialized to 0.
/*
#include <iostream>
#include <cstring>
using namespace std;

int main() {
 string s1, s2;
 cin >> s1 >> s2;

 int a[510] = {0}, b[510] = {0}, c[510] = {0};
 int len1 = s1.size(), len2 = s2.size();

 for (int i = 0; i < len1; i++)
 a[i] = s1[len1 - 1 - i] - '0';
 for (int i = 0; i < len2; i++)
 b[i] = s2[len2 - 1 - i] - '0';

 int len = max(len1, len2);
 int carry = 0;
 for (int i = 0; i < len; i++) {
 int sum = a[i] + b[i] + carry;
 c[i] = sum % 10;
 carry = sum / 10;
 }
 if (carry > 0) {
 c[len] = carry;
 len++;
 }

 for (int i = len - 1; i >= 0; i--)
 cout << c[i];
 cout << endl;

 return 0;
}
*/
```

Sample Input

```
100000000000000000000
1
```

Expected Output

```
100000000000000000001
```

例题 3：Consecutive Carries (999999 + 1)

```
// This tests the cascade carry scenario: every digit produces a carry.
// Uses the same code template. Walk through the loop:
// i=0: 9+1+0=10, c[0]=0, carry=1
// i=1: 9+0+1=10, c[1]=0, carry=1
// ... (all digits produce carry)
// After loop: carry=1, c[6]=1, len=7
// Output: 1000000
/*
#include <iostream>
#include <cstring>
using namespace std;

int main() {
 string s1, s2;
 cin >> s1 >> s2;

 int a[510] = {0}, b[510] = {0}, c[510] = {0};
 int len1 = s1.size(), len2 = s2.size();

 for (int i = 0; i < len1; i++)
 a[i] = s1[len1 - 1 - i] - '0';
 for (int i = 0; i < len2; i++)
 b[i] = s2[len2 - 1 - i] - '0';

 int len = max(len1, len2);
 int carry = 0;
 for (int i = 0; i < len; i++) {
 int sum = a[i] + b[i] + carry;
 c[i] = sum % 10;
 carry = sum / 10;
 }
 if (carry > 0) {
 c[len] = carry;
 len++;
 }

 for (int i = len - 1; i >= 0; i--)
 cout << c[i];
 cout << endl;

 return 0;
}
*/
```

Sample Input

```
999999
1
```

Expected Output

```
1000000
```